hw2-numbers: Sample Solutions

CSE20F26

In this assignment,

You will practice applying functions and tracing algorithms in multiple contexts, and exploring properties of positional number representations.

Relevant class material: Week 2 and Week 3.

Assigned questions

1. Functions and algorithms: in this question you'll explore the properties of the (integer) power and logarithm functions. We'll use some definitions we introduced in class and review quiz, namely the function b^i with domain $\mathbb{Z}^+ \times \mathbb{N}$ and codomain $\mathbb{N}$ defined recursively by

   Basis Step:
   $b^0 = 1$
   Recursive Step:
   If $i \in \mathbb{N}, b^{i+1} = b \cdot b^i$

   and the algorithm:

   Calculating integer part of base b logarithm

```
1   procedure logb(b,n: positive integers with b>1)
2   i := 0
3   while n > b-1
4     i := i+1
5     n := n div b
6   return i {i holds the integer part of the base b logarithm of n}
```

   (a) Choose a positive integer b between 3 and 6 (inclusive) and choose a nonnegative integer i between 2 and 5 (inclusive). Demonstrate how to calculate the result of the *logb* algorithm (procedure) when its input is the base b you chose and $n = b^i$ (for the i you choose.) A complete answer will include the specific choice of b and i, along with trace of the calculations of b^i and the computation of the algorithm, including (clear, correct, complete) calculations for each of the function applications and/or references to definitions and a trace table for algorithm that includes the values of all relevant variables at each iteration (See the annotated Week 2 notes and the review quiz for the level of detail expected in a trace of a function application and a trace table).

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (1)

Solution: Take $b = 4$ and $i = 3$ as an example:

$$\begin{aligned} n &= b^i \\ &= 4^3 \quad \text{(Substituting chosen values)} \\ &= 4 \cdot (4^2) \quad \text{(Recursive step, with } 3 = 2 + 1) \\ &= 4 \cdot (4 \cdot (4^1)) \quad \text{(Recursive step, with } 2 = 1 + 1\ ) \\ &= 4 \cdot (4 \cdot (4 \cdot (4^0)) \quad \text{(Recursive step (with } 1 = 0 + 1) \\ &= 4 \cdot (4 \cdot (4 \cdot (1))) \quad \text{(Basis step)} \\ &= 64 \end{aligned}$$

Trace for logb(4,64)				
	b	n	i	$n > b - 1$?
Initial Value	4	64	0	$64 > 3$? Yes
After 1 Iteration	4	16	1	$16 > 3$? Yes
After 2 Iteration	4	4	2	$4 > 3$? Yes
After 3 Iteration	4	1	**3**	$1 > 3$? No

Hence, the output of $logb(4, 64)$ is 3.

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (2)

(b) Now we'll go in the other direction: Demonstrate how to calculate the result of 7^y where y is the result of the *logb* algorithm (procedure) when its input is $b = 7$ and $n = 30$. A complete answer will include clearly labelled traces of the calculations of b^i and the computation of the algorithm, including (clear, correct, complete) calculations for each of the function applications and/or references to definitions and a trace table for algorithm that includes the values of all relevant variables at each iteration (See the annotated Week 2 notes for the level of detail expected in a trace of a function application and a trace table).

Solution:

Trace for logb(7,30)				
	b	n	i	$n > b - 1$?
Initial Value	7	30	0	$30 > 6$? Yes
After 1 Iteration	7	4	**1**	$4 > 6$? No

Hence, the output of $logb(7, 30)$ is 1.

$$\begin{aligned} b^i &= 7^1 \quad \text{(Substituting values)} \\ &= 7 \cdot (7^0) \quad \text{(Recursive step with } 1 = 1 + 0\text{)} \\ &= 7 \cdot (1) \quad \text{(Basis step)} \\ &= 7 \end{aligned}$$

(c) Logarithms and powers are supposed to "undo" one another. Explain whether your work in parts (a) and (b) supports that idea, whether you saw anything confusing or surprising about these calculations, and how to explain what you saw.

Solution: The numbers in part (a) support that idea as $4^3 = 64$ and $logb(4, 64) = 3$. However, the numbers in part (b) do not support that idea as $logb(7, 30) = 1$, while $7^1 = 7 \neq 30$. This happens because we are only "calculating integer part of base b logarithm" and throwing away the fractional part, resulting in $logb(b, n) = \lfloor \log_b n \rfloor$. Hence, $b^{logb(b,n)} \leq n$, and $b^{logb(b,n)} = n$ if and only if $\log_b n$ is an integer.

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (3)

2. Base expansions

(a) Pick an integer between 50 and 1000 (inclusive) that you (or one of your group members) came across at some point this week. In a sentence or two, give some context for why you're choosing this number). Write the base expansion of your chosen number in base 2 (binary), base 3 (ternary), base 4, and base 16 (hexadecimal).

Sample Solution: The number I pick is 53. I picked 53 because it is a prime number and I like prime numbers.

Trace for base2(53,2)						
	n	b	q	k	a_k	$q \neq 0$?
Initial Value	53	2	53	0		$525 \neq 0$? Yes
After 1 Iteration	53	2	26	1	$a_0 = 1$	$26 \neq 0$? Yes
After 2 Iteration	53	2	13	2	$a_1 = 0$	$13 \neq 0$? Yes
After 3 Iteration	53	2	6	3	$a_2 = 1$	$6 \neq 0$? Yes
After 4 Iteration	53	2	3	4	$a_3 = 0$	$3 \neq 0$? Yes
After 5 Iteration	53	2	1	5	$a_4 = 1$	$1 \neq 0$? Yes
After 6 Iteration	53	2	0	6	$a_5 = 1$	$0 \neq 0$? No

I conclude that $53 = (110101)_2$. Notice that, indeed,

$$1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 32 + 16 + 4 + 1 = 53$$

Trace for base2(53,3)						
	n	b	q	k	a_k	$q \neq 0$?
Initial Value	53	3	53	0		$53 \neq 0$? Yes
After 1 Iteration	53	3	17	1	$a_0 = 2$	$17 \neq 0$? Yes
After 2 Iteration	53	3	5	2	$a_1 = 2$	$5 \neq 0$? Yes
After 3 Iteration	53	3	1	3	$a_2 = 2$	$1 \neq 0$? Yes
After 4 Iteration	53	3	0	4	$a_3 = 1$	$0 \neq 0$? No

I conclude that $53 = (1222)_3$. Notice that, indeed,

$$1 \cdot 3^3 + 2 \cdot 3^2 + 2 \cdot 3^1 + 2 \cdot 3^0 = 27 + 18 + 6 + 2 = 53$$

Trace for base2(53,4)						
	n	b	q	k	a_k	$q \neq 0$?
Initial Value	53	4	53	0		$53 \neq 0$? Yes
After 1 Iteration	53	4	13	1	$a_0 = 1$	$13 \neq 0$? Yes
After 2 Iteration	53	4	3	2	$a_1 = 1$	$3 \neq 0$? Yes
After 3 Iteration	53	4	0	3	$a_2 = 3$	$0 \neq 0$? No

I conclude that $53 = (311)_4$. Notice that, indeed, $3 \cdot 4^2 + 1 \cdot 4^1 + 1 \cdot 4^0 = 48 + 4 + 1 = 53$.

Trace for base2(53,16)						
	n	b	q	k	a_k	$q \neq 0$?
Initial Value	53	16	53	0		$53 \neq 0$? Yes
After 1 Iteration	53	16	3	1	$a_0 = 5$	$3 \neq 0$? Yes
After 2 Iteration	53	16	0	4	$a_3 = 3$	$0 \neq 0$? No

I conclude that $53 = (35)_{16}$. Notice that, indeed, $3 \cdot 16 + 5 \cdot 16^0 = 48 + 5 = 53$.

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (4)

(b) What is the **smallest** width w in which they could write your chosen number in base 8 (octal) fixed-width w? Justify your answer with reference to the definitions of fixed-width expansions and relevant calculations.

Method 1: Run and trace the $base2(n, 8)$ (or $base1(n, 8)$) where n is the number you picked and claim that the minimum length w equals the length of the expression you got.

Example for Method 1:

Trace for base2(53,8)						
	n	b	q	k	a_k	$q \neq 0$?
Initial Value	53	8	53	0		$53 \neq 0$? Yes
After 1 Iteration	53	8	6	1	$a_0 = 5$	$3 \neq 0$? Yes
After 2 Iteration	53	8	0	2	$a_3 = 6$	$0 \neq 0$? No

$53 = (65)_8$, which means the minimum width in octal that can write 53 is 2 (as you can't fit 53 in one digit in octal).

Method 2: Instead of thinking smallest width w, we can also think about the alternative question: For a given width w, what is the maximum number that can be written in base 8 fixed-width w?
We know that $(a_{w-1} \dots a_0)_{8,w} = \sum_{i=0}^{w-1} a_i \cdot 8^i$, and $0 \leq a_i \leq 7$ for all i. It leads to:

$$\max((a_{w-1} \dots a_0)_{8,w}) = (77 \dots 7)_{8,w} = \sum_{i=0}^{w-1} 7 \cdot 8^i = 8^w - 1$$

Hence, we have:

w	$\max((a_{w-1} \dots a_0)_{8,w})$
1	7
2	63
3	511
4	4095
...	...
w	$8^w - 1$
...	...

And thus, returning to the question, if we take 53 as an example, $53 > 7$ means 53 **can't** be written in base 8 fixed-width 1; and $53 \leq 63$ means 53 **can** be written in base 8 fixed width 2, which means the minimum width to write 53 in octal is 2.
Following the same logic, we can conclude that:

range of numbers	w
$[1, 7]$	1
$[8, 63]$	2
$[64, 511]$	3
$[512, 4095]$	4
...	...
$[8^{w-1}, 8^w - 1]$	w
...	...

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (5)

(c) Express in roster method the set of numbers between 1 and 2 (exclusive) that be written without error (full precision) in binary fixed width expansion with integer part width 3 and fractional part width 2. Justify your answer with reference to the definitions of fixed-width expansions and relevant calculations.

Solution: As the set of numbers is between 1 and 2 (exclusive), we can conclude that the integer part must equal 1 (and its binary fixed-width 2 expansion must be $(001)_{2,3}$). For the 2 bits in the fractional part, we can choose from $\{01, 10, 11\}$ as 00 would result in 1.0 which is excluded. Hence, the set in binary representation would be

$$\{(001.01)_{2,3,2}, (001.10)_{2,3,2}, (001.11)_{2,3,2}\}$$

Which are $\{1.25, 1.5, 1.75\}$

(d) Consider the strings of 1s that have length 3, 5, 7, and 10. Calculate the numbers

$[111]_{s,3}$ $[111]_{2c,3}$
$[11111]_{s,5}$ $[11111]_{2c,5}$
$[1111111]_{s,7}$ $[1111111]_{2c,7}$
$[1111111111]_{s,10}$ $[1111111111]_{2c,10}$

Justify your answers with specific reference to the definitions of sign-magnitude and 2s complement expansions and relevant calculations and discuss any patterns you notice.

Solution:

$[111]_{s,3} = -(11)_2 = -(2+1) = -3$
$[11111]_{s,5} = -(1111)_2 = -(8+4+2+1) = -15$
$[1111111]_{s,7} = -(111111)_2 = -(32+\cdots+1) = -63$
$[1111111111]_{s,10} = -(111111111)_2 = -(256+\cdots+1) = -511$
$[111]_{2c,3} = -(2^{3-1} - (11)_2) = -(4-3) = -1$
$[11111]_{2c,5} = -(2^{5-1} - (1111)_2) = -(16-15) = -1$
$[1111111]_{2c,7} = -(2^{7-1} - (111111)_2) = -(64-63) = -1$
$[1111111111]_{2c,10} = -(2^{10-1} - (111111111)_2) = -(512-511) = -1$

The pattern is that: for all w,
$[11\ldots1]_{s,w} = -(11\ldots1)_{2,w-1} = -(2^{w-1}-1)$
$[11\ldots1]_{2c,w} = -(2^{w-1} - (11\ldots1)_{2,w-1}) = -(2^{w-1} - (2^{w-1}-1)) = -1$

Copyright Fall 2026 instructional team, **do not post publicly**, Version October 15, 2026 (6)

3. Multiple representations

Recall that, mathematically, a color can be represented as a 3-tuple (r, g, b) where r represents the red component, g the green component, b the blue component and where each of r, g, b must be from the collection $\{x \in \mathbb{N} \mid 0 \leq x \leq 255\}$. As an alternative representation, in this assignment we'll use base b fixed-width expansions to represent colors as individual numbers (this definition was introduced in this week's Review quiz).

Definition: A **hex color** is a nonnegative integer, n, that has a base 16 fixed-width 6 expansion

$$n = (r_1 r_2 g_1 g_2 b_1 b_2)_{16,6}$$

where $(r_1 r_2)_{16,2}$ is the red component, $(g_1 g_2)_{16,2}$ is the green component, and $(b_1 b_2)_{16,2}$ is the blue. For this question, let's call the set of hex colors H. In the Week 2 Review quiz (Question 3d), we explored a few different set builder definitions for H.

(a) Rewrite the set builder definition of a set below so that it refers to colors rather than numbers: $\{c \in H \mid c \mod 256 = 0\}$. A complete answer will justify the new set builder definition by connection with the definition of hex colors and how it impacts the colors that satisfy the specific property for this set.

Solution: We can write out $c = (r_1 r_2 g_1 g_2 b_1 b_2)_{16,6} = r_1 \cdot 16^5 + r_2 \cdot 16^4 + g_1 \cdot 16^3 + g_2 \cdot 16^2 + b_1 \cdot 16 + b_2$, which we can rewrite (by noticing $256 = 16^2$) as:

$$(r_1 \cdot 16^3 + r_2 \cdot 16^2 + g_1 \cdot 16 + g_2) \cdot (256) + (b_1 \cdot 16 + b_2)$$

$$\text{c div 256} \qquad\qquad \text{c mod 256}$$

The condition $c \mod 256 = 0$ implies that the color must follow the format $n = (r_1 r_2 g_1 g_2 00)$ as the last two digits must be $b_1 = b_2 = 0$ to make c mod 256 $= 16 \cdot b_1 + b_2 = 0$. We can rewrite the set builder with respect to color properties as follows:

$$\{c \in H \mid c \text{ has no blue component}\}$$

This means all colors in this set will be a combination of only red and green.

(b) Rewrite the set builder definition of a set below so that it refers to colors rather than numbers: $\{c \in HC \mid c < 65536\}$. A complete answer will justify the new set builder definition by connection with the definition of hex colors and how it impacts the colors that satisfy the specific property for this set.

Solution: We notice that $65536 = 256^2 = 16^4$ is $(010000)_{16,6}$ in base 16 fixed-with 6 representation. Hence, $c < 65536$ guarantees that $c < (010000)_{16,6}$, which in turn guarantees that $c \leq (00FFFF)_{16,6}$. Hence, the first two digits (r_1, r_2) must be 0 and the rest (the green and blue components) are not constrained. We can rewrite the set builder with respect to color properties as follows:

$$\{c \in HC \mid c \text{ has no red component}\}$$

(c) In art, we can mix two colors to get a new color. For example, red and blue make purple, blue and yellow make green, red and yellow make orange. A mathematical definition of **hex color mixing** would be a function with domain $HC \times HC$ and codomain HC where the result of applying this function to an input (n_1, n_2) is the hex color that results from mixing the hex colors n_1 and n_2.

Sample work for a related question that can be used as reference for the detail expected in your answer: Consider the attempted mathematical definition $f_1 : HC \times HC \to HC$ given by

$$f_1(\ (n_1, n_2)\ ) = n_1 + n_2$$

We will show that this function is not well-defined so it cannot give a hex color mixing definition. The reason it is not well-defined is that the application of the rule sometimes gives values that are not in the stated codomain. To see this, we need an example input to f_1 for which the output of the rule is not in HC. Here is one such example: $(n_1, n_2) = (\ (FFFFFF)_{16,6}, (FFFFFF)_{16,6}\ )$. This example is in $HC \times HC$ because $(FFFFFF)_{16,6}$ is a nonnegative integer that has a base 16 fixed-width 6 expansion so it is a hex color. We now apply the definition of the rule in f_1:

$$\begin{aligned} f_1(\ (n_1, n_2)\ ) &= f_1((\ (FFFFFF)_{16,6}, (FFFFFF)_{16,6}\ )) = (FFFFFF)_{16,6} + (FFFFFF)_{16,6} \\ &= 2\left(15 \cdot 16^5 + 15 \cdot 16^4 + 15 \cdot 16^3 + 15 \cdot 16^2 + 15 \cdot 16^1 + 15 \cdot 16^0\right) \\ &= 2 \cdot 15 \cdot (1048576 + 65536 + 4096 + 256 + 16 + 1) = 2 \cdot 15 \cdot 1118481 = 33554430 \end{aligned}$$

This is not a hex color because it is greater than or equal to $16^6 = 16777216$, so (using the Week 2 notes on which numbers can be represented with fixed-width expansions) it does not have a hexadecimal **fixed width** 6 expansion.

In this question, we'll look at another attempted mathematical definition for hex color mixing. We define $f_2 : HC \times HC \to HC$ given by

$$f_2(\ (n_1, n_2)\ ) = (n_1 + n_2)\ \textbf{div}\ 2$$

This is a well-defined function. You do not need to prove this or hand it in, but it's good practice to make sure you understand why it's well-defined. Notice that the function application

$$f_2(\ (\ (FF0000)_{16,6}, (FFFE00)_{16,6}\ )\ )$$

can be calculated as:

$$\begin{aligned} f_2(((FF0000)_{16,6}, (FFFE00)_{16,6})) &= ((FF0000)_{16,6} + (FFFE00)_{16,6}))\ \textbf{div}\ 2 \\ &= \Big(\ \left(15 \cdot 16^5 + 15 \cdot 16^4 + 0 \cdot 16^3 + 0 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0\right) \\ &\quad + \ \left((15 \cdot 16^5 + 15 \cdot 16^4 + 15 \cdot 16^3 + 14 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0\right)\ \Big)\ \textbf{div}\ 2 \\ &= (16711680 + 16776704)\ \textbf{div}\ 2 = 33488384\ \textbf{div}\ 2 = 16744192 \end{aligned}$$

Since this is less than $16^6 = 16777216$, we can represent it using base 16 fixed-width 6:.

$$(16744192)_{10} = 15 \cdot 16^5 + 15 \cdot 16^4 + 7 \cdot 16^3 + 15 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0 = (FF7F00)_{16,6}$$

Using the web tool `https://www.w3schools.com/colors/colors_rgb.asp`, we can verify that $(FF0000)_{16,6}$ is red, $(FFFE00)_{16,6}$ is yellow, and $(FF7F00)_{16,6}$ is orange.

Show that f_2 does not work as a hex color mixing definition by finding another ordered pair of hex colors for which the result of applying f_2 does not give the expected hex color. Include the numerical description of each colors you mention, alongside a description of them in English. Describe how you know what these colors are (if you use a web color tool, include its URL in your submission writeup; if not, describe your reasoning). Justify your example with (clear, correct, complete) calculations and/or references to definitions, and connecting them with the desired conclusion.

Solution: For this example, we're going to use the web color tool, found at `https://www.w3schools.com/colors/colors_rgb.asp`. Say we have the color $\#00FF00$, or pure green. If we also have the color $\#FF0000$, or pure red.



The colors $\#00FF00$ and $\#FF0000$ in order

We can follow the template of the calculation given above:

$$f_2(\ (\ (00FF00)_{16,6}, (FF0000)_{16,6}\ )\ )$$

can be calculated as:

$$\begin{aligned} f_2(((00FF00)_{16,6}, (FF0000)_{16,6})) &= ((00FF00)_{16,6} + (FF0000)_{16,6})) \ \textbf{div}\ 2 \\ &= \big(\ \big(0 \cdot 16^5 + 0 \cdot 16^4 + 15 \cdot 16^3 + 15 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0\big) \\ &\quad + \ \big((15 \cdot 16^5 + 15 \cdot 16^4 + 0 \cdot 16^3 + 0 \cdot 16^2 + 0 \cdot 16^1 + 0 \cdot 16^0\big)\ \big) \ \textbf{div}\ 2 \\ &= (65280 + 16711680) \ \textbf{div}\ 2 = 16776960 \ \textbf{div}\ 2 = 8388480 \end{aligned}$$

Since this is less than $16^6 = 16777216$, we can represent it using base 16 fixed-width 6:.

$$(8388480)_{10} = 7 \cdot 16^5 + 15 \cdot 16^4 + 15 \cdot 16^3 + 15 \cdot 16^2 + 8 \cdot 16^1 + 0 \cdot 16^0 = (7FFF80)_{16,6}$$

The colors $\#7FFF80$

When we use the web tool to see this color, it's a light green color rather than the brown we expect when we mix red and green, which shows that f_2 doesn't necessarily work as a hex color mixing definition as there exists an input that does not output the expected mix of colors.

4. Fixed-width addition.

(a) Choose example width 5 first summand and width 5 second summand so that in the binary fixed-width addition (adding one bit at time, using the usual column-by-column and carry arithmetic, and ignoring the carry from the leftmost column), the example satisfies all three conditions below simultaneously

(1) When interpreting each of the summands and the result in binary fixed-width 5, the result represents the actual value of the sum of the summands **and**

(2) when interpreting each of the summands and the sum in sign-magnitude width 5, the result represents the actual value of the sum of the summands **and**

(3) when interpreting each of the summands and the sum in 2s complement width 5, the result represents the actual value of the sum of the summands.

Solution:
Let the first summand be $(00011)_{2,5}$ and the second summand be $(00001)_{2,5}$. Computing the addition gets us

$$\begin{array}{cccccc} & & & 1 & 1 & \\ & 0 & 0 & 0 & 1 & 1 \\ + & 0 & 0 & 0 & 0 & 1 \\ \hline & 0 & 0 & 1 & 0 & 0 \end{array}$$

Now let's check each of the three conditions:

(1) Converting the summands from binary fixed-width 5 to decimal gives us $(00011)_{2,5} = 2 + 1 = 3$ and $(00001)_{2,5} = 1$. The sum $(00100)_{2,5} = 4 = 3 + 1$.

(2) With sign-magnitude, the left-most bit is the sign bit. In this case, both summands have a 0 as their left-most bit, so the summands are positive. Converting the summands from sign-magnitude width 5 to decimal gives us $[00011]_{s,5} = 1 \cdot 3 = 3$ and $[00001]_{s,5} = 1 \cdot 1 = 1$. The sum $[00100]_{s,5} = 4 = 3 + 1$.

(3) With 2s complement, the value denoted by the column of the left-most is subtracted from each summand. In this case, both summands have a 0 in that left-most position, so nothing is subtracted and the numbers are positive. Converting the summands from 2s complement width 5 gives us $[00011]_{2c,5} = 2 + 1 = 3$ and $[00001]_{2c,5} = 1$. The sum $[00100]_{2c,5} = 4 = 3 + 1$.

(b) Choose an example width 5 first summand and second summand so that in the binary fixed-width addition (adding one bit at time, using the usual column-by-column and carry arithmetic, and ignoring the carry from the leftmost column), the example satisfies all three conditions below simultaneously

(1) When interpreting each of the summands and the result in binary fixed-width 5, the result **does not** represent the actual value of the sum of the summands **and**

(2) when interpreting each of the summands and the sum in sign-magnitude width 5, the result **does not** represents the actual value of the sum of the summands **and**

(3) when interpreting each of the summands and the sum in 2s complement width 5, the result represents the actual value of the sum of the summands.

A complete solution will clearly specify each summand and the result of binary fixed-width addition with this choice of summands; will specify the value of each summand and the result for binary fixed-width 5, sign-magnitude width 5, and 2s complement width 5 (and include calculations connecting with the definitions of these representations to explain these values); and a conclusion connecting the calculations to the properties laid out in the question.

Solution:
Let the first summand be $(11001)_{2,5}$ and the second summand be $(01001)_{2,5}$. Computing the addition gets us

$$\begin{array}{cccccc} 1 & 1 & & & 1 & \\ & 1 & 1 & 0 & 0 & 1 \\ + & 0 & 1 & 0 & 0 & 1 \\ \hline & 0 & 0 & 0 & 1 & 0 \end{array}$$

Notice that we ignore the carry from the leftmost column. Now let's check each of the three conditions:

(1) Converting the summands from binary fixed-width 5 to decimal gives us $(11001)_{2,5} = 16 + 8 + 1 = 25$ and $(01001)_{2,5} = 8 + 1 = 9$. The sum $(00010)_{2,5} = 2 \neq 25 + 9$.

(2) In sign-magnitude, the left-most bit denotes the sign of the number. For the first summand, the left-most bit is 1, so its conversion to decimal is $[11001]_{s,5} = -1 \cdot (8 + 1) = -9$. For the second summand, the left-most bit is 0, so its conversion to decimal is $[01001]_{s,5} = 8 + 1 = 9$. The sum also has a 0 as its left-most bit, so its conversion to decimal is $[00010]_{s,5} = 2 \neq -9 + 9$

(3) In two's complement, the value denoted by the left-most bit is subtracted from the rest of the number. Since the first summand has a 1 in its left-most bit, its conversion to decimal is $[11001]_{2c,5} = -16 + 8 + 1 = -7$. Since the second summand has a 0 in its left-most bit, no subtraction is necessary. $[01001]_{2c,5} = 8 + 1 = 9$. The sum alsp has a 0 as its left-most bit, so its conversion to decimal is $[00010]_{s,5} = 2 = -7 + 9$.